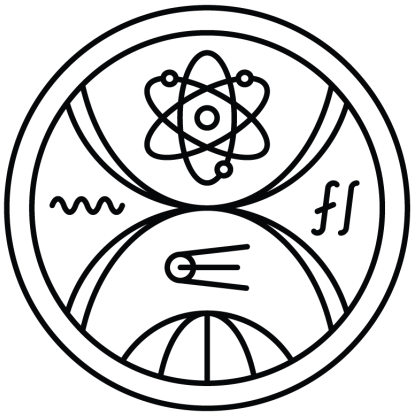
UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

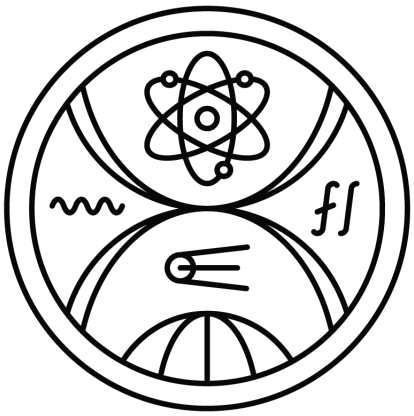


PODPORA INTELIGENTNÉHO RIADENIA ENERGETICKÝCH SIETÍ

DIPLOMOVÁ PRÁCA

2023
BC. OMAR AL-SHAFE´I

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY



PODPORA INTELIGENTNÉHO RIADENIA ENERGETICKÝCH SIETÍ

DIPLOMOVÁ PRÁCA

Študijný program:	Informatika
Študijný odbor:	Aplikovaná informatika
Školiace pracovisko:	Katedra aplikovanej informatiky
Školiteľ:	prof. RNDr. Mária Lucká, PhD.

Bratislava, 2023
Bc. Omar Al-Shafe´i





Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Bc. Omar Al-Shafe´i
Študijný program: aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: diplomová
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Podpora inteligentného riadenia energetických sietí
Support for intelligent management of smart energy networks

Anotácia: Stabilita inteligentnej energetickej siete a zabezpečenie dodávok elektrickej energie môže byť potenciálne narušená integráciou obnoviteľných zdrojov energie (napr. fotovoltických panelov), veľkých úložísk energie (batérií), ale aj významným rozvojom elektromobility. Tieto nové prvky sa v súčasnosti v čoraz väčšej miere stávajú súčasťou moderných inteligentných energetických sietí a preto ich integrácia do celej sústavy sa stala mimoriadne dôležitou. Pôvodne jednosmerná sieť, kde sa energia od veľkých výrobcov elektriny dodávala spotrebiteľom, sa postupne mení na obojsmernú sieť, v ktorej sa mnohí odberatelia - vďaka obnoviteľným zdrojom - stávajú súčasne aj drobnými výrobcami – prosumermi. Kvôli garancii stability celej sústavy a zabezpečeniu minimálnej ceny bolo potrebné vytvoriť tzv. agregátorov flexibitity, ktorým účastníci siete môžu poskytnúť svoju flexibilitu. Pod flexibilitou pritom rozumieme práva (1)na odber vyrobenej alebo uskladnenej elektrickej energie (napr. z fotovoltaiky, batérie alebo elektromobilu) v určitom čase a (2) obmedzenie spotreby (napr. vypnutie kúrenia).
Navrhnite a overte model práce agregátora flexibility, ktorý vďaka inteligentným algoritmom a dátovej analýze dokáže optimalizovať tok energie, zabezpečiť stabilné dodávky energie a minimalizovať náklady spotrebiteľov. Svoje riešenie obmedzte pre vybraný typ prosumerov/spotrebičov. Použite pritom vhodné optimalizačné metódy, predovšetkým metódy strojového učenia. Svoje riešenie implementujte a porovnajte s existujúcimi riešeniami na dostupných dátach.

Literatúra:
1. Li, T., Sun, B., Chen, Y., Ye, Z., Low, S. H., & Wierman, A. (2020). Real-time Aggregate Flexibility via Reinforcement Learning. 1–17. http://arxiv.org/abs/2012.11261
2. Fernando Lezama, Joao Soares, Bruno Canizes, Zita Vale: Flexibility management model of home appliances to support DSO requests in smart grids, Sustainable Cities and Society, Volume 55, 2020, ISSN 2210-6707, https://doi.org/10.1016/j.scs.2020.102048
3. Tongxin Li and Bo Sun and Yue Chen and Zixin Ye and Steven H. Low and Adam Wierman: Learning-Based Predictive Control via Real-Time Aggregate Flexibility, IEEE Transactions on Smart Grid, 12 (6), 2021, 97--4913, https://doi.org/10.1109%2Ftsg.2021.3094719.





Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

4. Steve Wattam: Artificial intelligence and machine learning approaches to energy demand-side response: A systematic review, Renewable and Sustainable Energy Reviews, Volume 130, 2020, 109899, ISSN 1364-0321, https://doi.org/10.1016/j.rser.2020.109899.

Vedúci: prof. RNDr. Mária Lucká, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.

Dátum zadania: 13.12.2022

Dátum schválenia: 13.12.2022 prof. RNDr. Roman Ďurikovič, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

Poďakovanie: Chcel by som poďakovať mojej školiteľke za cenné rady počas tvorby diplomovej práce.

Abstrakt

Slovenský abstrakt v rozsahu 100-500 slov, jeden odstavec. Abstrakt stručne sumarizuje výsledky práce. Mal by byť pochopiteľný pre bežného informatika. Nemal by teda využívať skratky, termíny alebo označenie zavedené v práci, okrem tých, ktoré sú všeobecne známe.

Kľúčové slová: jedno, druhé, tretie (prípadne štvrté, piate)

Abstract

Abstract in the English language (translation of the abstract in the Slovak language).

Keywords:

Obsah

Úvod **1**

1 Súčasný stav riešenej problematiky **4**
1.1 Model systému 7
1.2 Problém, ktorý riešime 9

2 Návrh nášho riešenia **10**
2.1 Maximálna entropická odozva 10
2.2 Proces učenia 12
2.2.1 Aproximovanie maximálnej entropickej odozvy 13
2.2.2 Odmeňovacia funkcia 14
2.3 Online penalizované prediktívné riadenie 15
2.4 Systém uzavretej slučky 16
2.5 Proces testovania 17
2.6 Jednoduché online plánovacie algoritmy 18
2.6.1 Least-Laxity-First 18
2.6.2 Smoothed Least-Laxity-First 20
2.7 Offline optimálny algoritmus 22

3 Dáta **24**
3.1 Forma dát 24

4 Experimenty **26**
4.1 Metrika merania porušení obmedzení systému 27
4.2 Hyperparametre v experimentoch 27
4.3 Opis experimentov 29

Záver **32**

Príloha A **37**

Príloha B **38**

Zoznam obrázkov

1.1 Model systému. 9

2.1 Príklad maximálnej entropickej odozvy. 12
2.2 Pseudokód online algoritmu PPC. 15
2.3 Pseudokód systému uzavretej slučky. 17
2.4 Pseudokód algoritmu LLF pre jeden časový krok. 19
2.5 Pseudokód bisekčnej metódy používanej v algoritme LLF 19
2.6 Pseudokód algoritmu sLLF pre jeden časový krok. 21
2.7 Kód bisekčnej metódy používanej v algoritme sLLF. 22

3.1 Príklad vstupných dát. 24

4.1 Ukážka nabíjania v nabíjacej stanici Caltech 29
4.2 Ukážka nabíjania v nabíjacej stanici jpl 30
4.3 Ukážka nabíjania z dát caltechovskej nabíjacej stanice 30
4.4 Ukážka nabíjania z dát caltechovskej nabíjacej stanice 31
4.5 Ukážka nabíjania z dát caltechovskej nabíjacej stanice 31

Zoznam tabuliek

1.1 Tabuľka notácií. 6

3.1 Opis parametrov z dátovej množiny ACN-Data. 25

Úvod

Energetická sieť je sieť infraštruktúry, vďaka ktorej môže distribútor elektrickej energie dodávať energiu z elektrární spotrebiteľom. Inteligentná energetická sieť je typ energetickej siete, ktorá používa aktuálne komunikačné technológie, pokročilé senzory, analýzy, monitorovanie a riadenie tokov elektrickej energie [1]. Inteligentné merače sú jednou z najdôležitejších prvkov inteligentných energetických sietí. Tieto merače merajú spotrebu elektrickej energie jej spotrebiteľov a následne tieto údaj posielajú dodávateľovi elektrickej energie v krátkych intervaloch, a to v rozmedzí od 15 minút do 30 minút. Vďaka týmto údajom môžeme predpovedať spotrebu elektrickej energie v budúcnosti s veľkou presnosťou. Pomocou týchto predikcií spotreby energie dokáže inteligentná energetická sieť riadiť výrobu elektrickej energie v reálnom čase [2, 3].

Vzhľadom na energetickú krízu, na zintenzívnenie snahy EU o zníženie uhlovodíkovej stopy, a na klimatické zmeny, ktorých sme svedkami, rastie snaha o integráciu obnoviteľných zdrojov (veterné a fotovoltické elektrárne) do energetických systémov. Pre distribútora elektrickej energie (na Slovensku sa nazýva SEPS) má veľký význam stabilita siete, stabilita dodávok elektrickej energie bez výpadkov a príprava na prechod zo štandardných výrobných zdrojov energie na obnoviteľné zdroje energie. Nestabilitu do energetickej siete vnášajú práve obnoviteľné zdroje, ktoré vyrábajú elektrickú energiu v závislosti od počasia, a takto vyrobenú elektrickú energiu vo všeobecnosti nemožno skladovať, aj keď sa časť z nej dá uložiť do rôznych úložísk, a to napr. do (lítiových) batérií, do elektrických vozidiel [4, 1, 5].

Komunikačné technológie inteligentnej energetickej siete umožnujú obojsmerný tok energie medzi dodávateľmi a spotrebiteľmi tejto energie narozdiel od tradičných energetických sietí, ktoré poskytujú výlučne jednosmerný tok energie od dodávateľov k spotrebiteľom. Inteligentné energetické sieťe majú schopnosť integrovať obnoviteľné zdroje energie, a tým nahradiť používanie fosilných palív vďaka svojej vylepšenej inteligencii a odolnosti [1].

V závislosti od spotreby elektrickej energie kolíše jej cena podľa vopred dohodnutých taríf, ktoré zohľadňujú výšku výroby a spotreby počas dňa. Najvýhodnejším riešením je posúvať spotrebu elektrickej energie na čas, kedy je cena najnižšia a spotrebuvávať ju najviac vtedy, keď je energia najlacnejšia [6, 7]. Riadiť spotrebu môže sám spotrebiteľ energie alebo môže túto právomoc delegovať agregátorovi flexibity. Agregátor flexibility totiž vie, koľko energie má k dispozícii, resp. koľko potrebuje kúpiť na trhu s elektrickou energiou, t. j. na

spotovom trhu. Jeho miesto je medzi spotrebiteľom a distribútorom (správcom celej sústavy - na Slovensku sa nazýva SEPS) [8].

Úloha agregátora flexibility zatiaľ nie je presne definovaná. Táto úloha agrégatora môže byť určená treťou stranou alebo už učinkujúcim hráčom na trhu s elektrickou energiou. Nezávislý agregátor môže poskytovať jednu službu alebo viacero služieb. Medzi služby agregátora patria združovanie flexibility, dodávky energie spotrebiteľom, ako aj vyváženie dopytu a ponuky na trhoch s elektrickou energiou [9].

Vlastníci elektromobilov môžu hrať rolu prosmerov, ak sa zapájajú do programov o vyvážení dopytu. Prosumeri môžu napríklad poskytovať nabíjacej sieti energiu za predpokladu, že technológie umožnujú obojsmerné nabíjanie. Participácia prosumerov benefituje prosumerov tým, že dostanú odmeny za poskytovanie energie. Nabíjacia sieť benefituje tým, že je menej odkázaná generovať energiu počas najvyššieho dopytu po energií pomocou nákladových generátorov, ako napríklad plynových turbín [10].

Jednou z úloh agregátora flexibility je aj úloha minimalizácie nákladov na elektrickú energiu v spoločných garážach, kde sa nachádza určité množstvo nabíjacích staníc. Inšpirovaní článkami [11, 12, 13, 14, 15], sme sa zamerali na hľadanie optimalizácie nabíjania elektrických vozidiel tak, aby si každé elektrické vozidlo dobilo dostatok energie, nebol porušený maximálný možný príkon (spotrebovaná energia) v čase, a zároveň sa minimalizovala cena dobitia.

V našom riešení uvažujeme interakciu agregátora a nezávislého systémového operátora v systéme uzavretej slučky (opísaný v druhej kapitole), aby sme vyriešili problém optimalizácie nabíjania, ktorý sme spomenuli v predchádzajúcom odseku. Agregátor privileguje úlohu optimalizácie nabíjania nezávislému systémovému operátorovi (podľa článku [11] agregátor nemusí mať dostatok sily, aby sám vyriešil optimalizačný problém). Systémový operátor vie minimalizovať svoje operačné náklady, optimalizovať nabíjanie elektromobilov, alebo dosiahnuť iné svoje ciele na základe získanej (dostatočne informatívnej) spätnej odozvy o flexibilite od agregátora. Signál spätnej odozvy o flexibilite poskytuje informácie o agregovanej flexibilite agregátora. Agregovaná flexibilita agregátora je sumárom obmedzení systému. Dizajn signálu spätnej odozvy o flexibilite je komplexný problém. My používame v našom riešení signál spätnej odozvy o flexibilite, ktorý nazývame maximálna entropická odozva.

Nezávislý systémový operátor posiela kúpenú časovo meniacu sa energiu (energiu, ktorá je riešením optimalizačného problému a spľňa operačné obmedzenia) agregátorovi, z ktorej by agregátor mal čo najviac využiť na nabíjanie flotily elektromobilov. Kalifornský nezávislý systémový operátor je príkladom systémového operátora, ktorý zabezpečuje posúvanie spotreby energie v čase, skladovanie energie [11, 13, 16].

V kapitole 1 opisujeme stručne existujúce riešenia problematiky optimalizácie nabíjania, ktoré používajú rozdielne prístupy na riešenie problematiky. Spomíname aj tabuľku notácií a aj opis modelu systému, ktorý používame pri riešení problému. V kapitole 2 opisujeme návrh riešenia problému, opis a vysvetlenie použitých algoritmov, prípadne aj definície alebo

vysvetlenie premenných, ktoré sa v daných algoritmov vyskytujú. V kapitole 3 sa venujeme opisu dátovej množiny, z ktorej vychádzame. V kapitole 4 opisujeme aké nastavenia v algoritmoch sme si zvolili na vykonávanie experimentov a aj výsledky experimentov a diskusia o výsledkoch.

1 Súčasný stav riešenej problematiky

V tejto kapitole opisujeme súčasné riešenia podobnej problematiky, z ktorých niektoré vychádzajú z dátovej množiny ACN-Data (viac o dátovej množine ACN-Data sa nachádza v kapitole 3), pomocou ktorej sú schopní testovať výkonnosť plánovacích algoritmov. Uvádzame jednotlivé prístupy, ktoré si autori článkov zvolili na riešenie podobnej alebo rovnakej problematiky. Opisujeme aj jednotlivé riešenia od autorov a výsledky testovania algoritmov, ktoré predstavili.

Algoritmy, ktoré riešia problematiku optimalizácie nabíjania elektromobilov spadajú (na základe článku [14]) do dvoch kategórii

1. Online algoritmy riešia problém optimalizácie nabíjania elektromobilov pomocou metód, ktoré rozhodujú o pláne nabíjania elektromobilov len na základe o informácií o nabíjaných elektromobiloch po aktuálny čas.

2. Offline algoritmy riešia problém optimalizácie nabíjania elektromobilov pomocou metód, ktoré rozhodujú o pláne nabíjania elektromobilov na základe kompletných informácií o všetkých elektromobiloch.

Na riešenie problematiky optimalizácie nabíjania elektromobilov si výskumníci zvolili prístupy modelového prediktívneho riadenia, učenia posilovaním a dynamického programovania. Tradičné algoritmy ako least-laxity-first a earliest-deadline-first nie sú schopné sami riešiť túto problematiku, lebo ich hlavným cieľom je maximalizovať množstvo nabitej energie (t. j. zameriavajú sa len na úspešné dobitie elektromobilov). V tejto kapitole opisujeme hlavne online algoritmy na riešenie problematiky optimalizácie nabíjania elektromobilov. [17]

Autori článku [18] predstavujú online adaptívny plánovací algoritmus založený na modelovom prediktívnom riadení a metódach konvexnej optimalizácie. Porovnania adaptívne plánovacieho algoritmu s algoritmami least-laxity-first, Round Robin a earliest-deadline-first ukázali, že adaptívny plánovací algoritmus dokáže zabezpečiť dodanie väčšieho množštva požadovanej energie elektrickým vozidlám ako algoritmy v nabíjacej sieti s nízkou kapacitou na energiu. Na ďalšom príklade ukázali, že adaptívny plánovací algoritmus dokáže minimalizovať náklady spotrebiteľov na spotrebovanú energiu.

Autori článku [19] implementujú dvojfázovú nabíjaciu schému založenej na transakčnej riadení. Agregátor vykonáva nabíjanie elektromobilov v reálnom čase, aby minimalizoval

svoje celkové operačné náklady. Predstavujú algoritmus založený na modelovom prediktívnom riadení, pomcou ktorého agregátor vie dokončiť všetky transakcie na transakčnom trhu. Agregátor sa rozhoduje o pláne nabíjania elektromobilov na základe dokončených transakcií na trhu, ktoré pozostávajú z časových požiadaviek a preferencií používateľov elektromobilov. Verifikovali funkčnosť navrhnutej schémy na svojich experimentoch.

V článku [20] autori predstavujú dvojfázový aproximačný algoritmus dynamického programovania s cieľom dodať elektrickým vozidlám ich požadovanú energiu a minimalizovať náklady za nabíjanie elektrických vozidiel na parkoviskách komerčných budov. Vstupné dáta o prichádzajúcich elektrických vozidlách modelujú na základe Poissonovej distribúcie. Na zistenie funkčnosti ich algoritmu porovnali ceny nabíjania pri nabíjaní pomocou ich algoritmu s cenou nabíjania pomocou štandardného aproximačného algoritmu. Zistili, že ich algoritmus v porovnaní so štandardným aproximačným algoritmom dynamického programovania dosahuje v priemere o 2.5% nižšiu cenu nabíjania elektrických vozidiel.

Autori článku [21] predstavujú metódu založenú na hlbokom učení posilňovaním (ktorá používa algoritmus hlbokého učenia posilňovaním DDPG) na riešenie problému minimalizácie cien za dobitie elektromobilov. Na riešenie problému najskôr použijú vylepšenú verziu LSTM neurónovej siete nazývajúcu sa JANET na to, aby získali (extrahovali) vlastnosti cien elektriny. Potom aplikovaním navrhovanej metódy vedia vyriešiť problém optimalizácie nabíjania, t. j. nastaviť stratégiu nabíjania na základe extrahovaných vlastností cien elektriny tak, že sa zníži cena za nabíjanie energie pre vodičov elektromobilov. Výkonnosť navrhovanej metódy otestovali na simulovaných experimentoch. Z testov na simulovaných experimentoch zistili, že navrhovaná metóda môže znížiť cenu za nabíjanie elektromobilov až o 70.2% v porovnaní s inými metódami.

V článku [22] autori riešia problém minimalizácie nákladov za nabíjanie elektromobilov v solárnej nabíjacej stanici na základe modifikovaného algoritmu dynamického programovania. Algoritmus otestovali na štyroch prípadoch. Výsledky ukázali, že s použitím algoritmu modifikovaného dynamického programovania je možné značne zredukovať náklady solárnej nabíjacej stanice na energiu (od 6% do 30.7%), čo indikuje nižšie ceny nabíjania pre vlastníkov elektromobilov.

Autori článku [11] prezentujú vylepšenú verziu nášeho riešenia problému optimalizácie nabíjania elektromobilov. Predpokladajú, že systémový operátor má pefektné predikcie cenových funkcií a spätných odoziev o flexibilite w časových krokov t. j. o aktuálnom časovom kroku a budúcich. Ukázali, že pre ich riešenie problému čím je w vyššie, tým viac výsledne riešenie minimalizuje systémové náklady. Riešenie testovali pre premmenú w od 1 do 4. Riešenia s premmenou w vyššou ako 4 mali horší výsledok, lebo negenerovali presnú spätnú odozvu o flexibilte. Odmena vrátená počas trénovacej fáze pomocou algoritmu učenia posilňovaním SAC nekonvergovala veľmi dobre.

V tabuľke 1.1 prezentujeme notáciu, ktorú používame v ďalšom texte, ak nie je explicitne uvedené inak.

notácia	vysvetlenie
$\mathscr{T}$	Množina diskrétneho času systému
K	Množina všetkých nabíjačiek v nabíjacej stanici
N	Množina všetkých obsluhovaných elektromobilov
N'	Množina aktívnych (pripojených) elektromobilov
X	Množina realizovateľných stavov
X_t	Množina realizovateľných stavov v čase t
U	Množina realizovateľných akcií
U_t	Množina realizovateľných akcií v čase t
$a(i)$	Čas príchodu (pripojenia) elektromobilu i
$d(i)$	Čas odchodu (odpojenia) elektromobilu i
$e(i)$	Množstvo požadovanej energie elektromobilu i (jednotka: kWh)
$r(i)$	Maximálna rýchlosť nabíjania elektromobilu i (jednotka: kW)
$d_t(i)$	Zostávajúci čas nabíjania elektromobilu i (jednotka: hod)
$e_t(i)$	Zostávajúca požadovaná energia elektromobilu i v čase t (unit: kW)
x_t	Stav agregátora v čase t
u_t	Realizovaná akcia v čase t
f_t	Deterministická prechodová funkcia v čase t
π	Jednoduchý plánovací algoritmus
s_t	Plán nabíjania pre čas t
$s_t(i)$	Plán nabíjania elektromobilu i v čase t
Δ	Časová jednotka (jednotka: hodiny)
p_t	Agregovaná spätná odozva o flexibilite v reálnom čase t
c_t	Cenová funkcia v čase t
C_t	Kumulatívna náklady systému v čase t
Ψ_{SAC}	Agregátorova funkcia
$\|\|x\|\|_2$	l_2-norma vektora x
$\|x\|_+$	$max\{0,x\}, x \in R$
$\|x\|_a^b$	Projekcia skalára x na interval $[a,b]$
$l_t(i)$	Laxita elektromobilu i v čase $t \in \mathscr{T}$
$I(\cdot)$	Indikátorova funkcia,ak je boolovský výraz vo vnútri pravdivý, vráti 1, inak 0
$H(\cdot)$	Diskrétna entropická funkcia

Tabuľka 1.1: Tabuľka notácií. Zdroje tabuľky sú [11, 12, 17, 14, 23].

Náhodné premenné označujeme veľkými písmenami ako napríklad X, U. Realizácie náhodných premenných označujeme malými písmenami ako napríklad u. Základ logaritmu log je prirodzené číslo e, ak nie je uvedené inak. Premenná u_t je vektor euklidovskom priestore. Zlúčenie dvoch sekvencií u a v označujeme pomocou zátvorky (u,v). Na zjednodušenie zápisu premennú p_t nazývame v ďalšom texte ako spätná odozva o flexibilite [11].

1.1 Model systému

Model systému, ktorý v tejto sekcií definujeme a používame je kombinácia modelov systému z publikácii [12, 15, 14, 11]. Definície, predpoklady a opisy v tejto sekcií čerpáme z článkov [12, 15, 14, 11].

Uvažujeme systém s jednou nabíjacou stanicou s $k \in K = \{1,\dots,54\}$ nabíjačkami, ktoré riadi agregátor. Nabíjačky obsluhujú flotilu elektromobilov $i \in N = \{1,2,3,\dots\}$. Používame diskrétny časový model, v ktorom je čas rozdelený na intervaly rovnakých vzorkovaní s indexom $t \in \mathscr{T} = \{1,\dots,T\}$. Každý elektromobil $i \in N$ prichádza na nabíjaciu stanicu (pripája sa na voľnú nabíjačku) v čase $a(i)$ s požadovanou energiou $e(i)$ (jednotka: kWh), maximálnou rýchlosťou nabíjania $r(i)$ (jednotka: kW) a s časom odchodu (čas odpojenia od nabíjačky) $d(i)$. Časové hodnoty premenných $a(i)$ a $d(i)$ sú normalizované na základe intervaloch rovakých vzorkovaní $\mathscr{T}$. Platí, že $1 \leq a(i) \leq T$ a $1 \leq d(i) \leq T$. Predpokladáme, že je dostatočné množstvo nabíjačiek na obsluhovanie elektromobilov vo všetkých časoch $t \in \mathscr{T}$. To znamená, že každý elektromobil $i \in N$ sa vždy pripojí na voľnú nabíjačku v čase jeho príchodu $a(i)$ a odpojí sa od danej nabíjačky v čase odchodu $d(i)$. Agregátor rozhoduje o dodávaní energie elektromobilov.

V čase $t \in \mathscr{T}$ agregátor dodá $s_t(i) \in R^+$ energie elektromobilu i. Množina R^+ je množinou nezáporných reálnych čísel. Hodnoty $s_t(i) \in R^+, i \in N$ vypočíta agregátor na základe jednoduchého plánovacieho algoritmu π a pomocou signálu $u_t \in U$ získaného od systémového operátora. Jednoduché plánovacie algoritmy, ktoré v ďalšom texte opisujeme sú least-laxity-first, earliest-deadline-first a smoothed least-laxity-first. Signál u_t sa generuje v každom čase $t \in \mathscr{T}$ z diskrétnej a uzavretej domény $U \subseteq R$. Dodaním energie u_t elektromobilom a aktualizovaním ich zostávajúceho času nabíjania agregátor zmení svoj stav z x_t na x_{t+1}. Stav agregátora x_t v čase t je kolekcia n dvojíc $(d_t(i), e_t(i) | a(i) \leq t \leq d(i), i \in N)$ v stavovom priestore $X \subseteq R^{2n}$. Premenná $d_t(i)$ vyjadruje zostávajúci čas nabíjania elektromobilu i v čase t a premenná $e_t(i)$ vyjadruje množstvo zostávajúcej požadovanej energie elektromobilu i v čase t.

Potom agregátor pomocou funkcie ψ_{SAC} vygeneruje spätnú väzbu o flexibilite p_{t+1} na základe svojho stavu x_{t+1}. Systémový operátor v každom čase t na základe získanej spätnej odozvu o flexibilite p_t od agregátora a na základe a ceny energie c_t vygeneruje také u_t, ktoré je riešením optimalizačného problému, ktorý riešime (optimalizačný problém definujeme v sekcií 1.2). Viac informácii o interakcii medzi agregátorom a systémovým operátorom, učení a generovaní spätnej odozvy o flexibilite a generovaní signálu u_t je v kapitole 2. Platí, že $s_t = \pi(u_t, x_t) = (s_t(i) | i \in N)$. Na základe pridelenej energie s_t elektromobilom $i \in N$ sa v každom čase $t \in \mathscr{T}$ aktualizuje stav systému

$$x_{t+1} = f_t(x_t, u_t), x_t \in X_t(x_{<t}, u_{<t}), u_t \in U_t(x_{<t}, u_{<t}), \tag{1.1}$$

kde f_t je deterministická prechodová funkcia a množiny $X_t(x_{<t}, u_{<t})$ a $U_t(x_{<t}, u_{<t})$ sú ob-

medzenia systému. Obmedzenia $X_t(x_{<t}, u_{<t})$ a $U_t(x_{<t}, u_{<t})$ sú závislé od predchádzajúcich stavov $x_{<t} = (x_1, \ldots, x_{t-1})$ a akcií $u_{<t} = (u_1, \ldots, u_{t-1})$. Stav agregátora x_t a akcia u_t sa musia vyberať z množín X_t a U_t. Pre zjednodušenie zápisu premenných budeme označovať v ďalšom texte X_t namiesto $X_t(x_{<t}, u_{<t})$ a U_t namiesto $U_t(x_{<t}, u_{<t})$. Obmedzenia na plány nabíjania $s_t, t = 1, \ldots, T$ sú

$$s_t(i) = 0, t < a(i), i \in N, \tag{1.2}$$

$$s_t(i) = 0, t > d(i), i \in N, \tag{1.3}$$

$$\sum_{i \in N} s_t(i) = u_t, t = 1, \ldots, T, \tag{1.4}$$

$$\sum_{t=1}^{T} s_t(i) = e(i), i \in N, \tag{1.5}$$

$$0 \leq s_t(i) \leq r(i), t = 1, \ldots, T. \tag{1.6}$$

Obmedzenie (1.2) zabezpečuje, že elektromobily nie sú nabíjané pred ich pripojením sa na nabíjačky. Analogicky, obmedzenie (1.3) zabezpečuje, že elektromobily nie sú nabíjané po ich odpojení z nabíjačiek. Zavádzame obmedzenie (1.4), lebo agregátor by mal vedieť využiť celú získanú energiu u_t na nabíjanie elektromobilov v čase t. Obmedzenie (1.5) garantuje, že agregátor dodá požadovanú energiu elektromobilom do konca ich nabíjania. Obmedzenie (1.6) garantuje, že každý elektromobil i sa nabíja po svoju maximálnu rýchlosť nabíjania $r(i)$. Obmedzenia (1.2), (1.3) a (1.6) sú vždy splnené pomocou jednoduchého plánovacieho algoritmu π. Jediné obmedzenia, ktoré môžu byť porušené sú (1.4) a (1.5). Obmedzenie (1.4) je porušené, ak $\exists t \in \mathscr{T} : u_t > \sum_{i \in N} s_t(i)$ a obmedzenie (1.5) je porušené, ak $\exists i \in N : \sum_{t=1}^{T} s_t(i) < e(i)$. Dôvod, prečo môžu byť porušenia porušené sa nachádza na konci podsekcie 2.2.1.

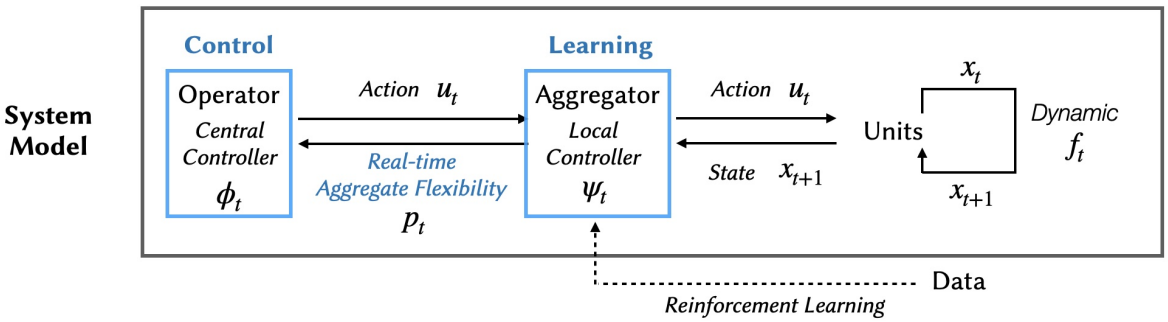
Predpokladáme, že cenové funkcie $c_1, \ldots, c_T$ sú dostupné len systémovému operátorovi. Cenová funkciu v čase t je zobrazenie $c_t(\cdot) : U \to R^+$. Zavádzame podmienku pre hladkosť cenovej funkcie. Predpokladáme, že každá cenová funkcia musí byť Lipschitzovsky spojitá.

To znamená, že pre každý čas t cenová funkcia c_t je Lipschitzovsky spojitá. Predpodkladáme, že $\exists L_c > 0$ také, že

$$|c_t(u_1) - c_t(u_2)| \leq L_c ||u_1 - u_2||_2, \forall u_1, u_2 \in U, t \in \mathscr{T},$$

kde L_c Lipschitzovská konštanta. Pre daný vektor x je l_2-norma vektora x (definovaná v [24]) $||x||_2 = (|x_1|^p + \cdots + |x_n|^p)^{\frac{1}{p}}$.

Predpokladáme, že obmedzenie X_t je borelovská množina v R^n a obmedzenie U_t je borelovská množiny v R. Predpokladáme aj, že množina $U \subseteq R$ je uzavretá a ohraničená. Deterministická prechodová funkcia $f_t(\cdot, \cdot) : X_t \, x \, U_t \to X_{t+1}$ je borelovský spočítateľná funkcia pre každé $t \in \mathscr{T}$.



Obr. 1.1: Model systému (prevzatý z [12]), z ktorého vychádzame. Namiesto označenia Ψ_t používame označenie Ψ_{SAC} a namiesto označenia ϕ_t používame označenie PPC.

1.2 Problém, ktorý riešime

Našim cieľom je vyriešiť online verziu offline optimalizačného problému reálnom čase pomocou koordinácie agregátora a systémového operátora

$$\min_{u_1,\ldots,u_T} C_T(u_1,\ldots,u_T) \tag{1.7}$$

vzhľadom na $\forall t = 1,\ldots,T$:

$$x_{t+1} = f_t(x_t,u_t), \tag{1.8}$$

$$x_t \in X_t, \tag{1.9}$$

$$u_t \in U_t, \tag{1.10}$$

kde $C_t(u_1,\ldots,u_T) = \sum_{t=1}^{T} c_t(u_t) = \sum_{t=1}^{T} c_t u_t$ sú kumulatívne náklady systému za energiu. Agregátor zabezpečuje dodržanie obmedzení systému (1.8)-(1.10) a systémový operátor vykonáva minimalizáciu svojich kumulatívných nákladov $C_T(u_1,\ldots,u_T)$. Tento optimalizačný problém je generický formulovaný. My riešime špecificky online verziu offline optimalizačného problému

$$\min_{u_1,\ldots,u_T} C_T(u_1,\ldots,u_T) \tag{1.11}$$

vzhľadom na:

$$(1.2)-(1.6), \tag{1.12}$$

$$e_t(i) = e_{t-1}(i) - s_t(i),\ i \in N, t = 1,\ldots,T, \tag{1.13}$$

$$d_t(i) = d_{t-1}(i) - \Delta,\ i \in N, t = 1,\ldots,T, \tag{1.14}$$

kde (1.13) a (1.14) sú konkrétne príklady prechodových funkcií $f_1,\ldots,f_T$. Premenná Δ je čas medzi dvoma po sebe idúcimi časovými krokmi. Vyriešením optimalizačného problému (1.11)-(1.14) dokážeme optimalizovať nabíjanie tak, ako sme spomínali v úvode, lebo energia u_t je v tomto probléme definovaná ako súčet nabitej energie v čase t (platí pre $t = 1,\ldots,T$).

2 Návrh nášho riešenia

V práci porovnávame niekoľko algoritmov, ktoré riešia optimalizačný problém (1.11)-(1.14). Špecificky porovnávame algoritmy penalizovaného prediktívneho riadenia a offline optimálny algoritmus. Offline optimálny algoritmus opisujeme v sekcii 2.7. Pri použití algoritmu penalizovaného prediktívneho riadenia dokáže agregátor nabíjať pomocou rôznych jednoduchých plánovacích algoritmov. V tejto kapitole opisujeme systém uzavretej slučky (v sekcii 2.4) ktorý rieši optimalizačný problém (1.11)-(1.14) pomocou penalizovaného prediktívneho riadenia (opis v sekcii 2.3), generovanej spätnej odozvy o flexibilite p_t a jednoduchému plánovaciemu algoritmu. Jednoduché plánovacie algoritmy opisujeme v sekcii 2.6. Generovaná spätná odozva o flexibilite p_t je aproximáciou maximálnej entropickej odozvy, ktorú definujeme a opisujeme 2.1. Maximálnu entropickú odozvu, ktorá je založená na maximalizácii entropie sa dá aproximovať algoritmami strojového učenia, my používame algoritmus učenia posilňovaním. Proces učenia, opis algoritmu učenia posilňovaním a opis vhodnej odmeňovacej funkcie je v sekcii 2.2. Proces testovania (ktorý opisujeme v sekcii 2.5) sa odlišuje od procesu učenia tým, že sa už generuje naučená maximálna entropická odozva [12, 11].

2.1 Maximálna entropická odozva

V tejto sekcii najprv opisujeme dizajn a vlastnosti spätnej odozvy o flexibilite, ktorú nazývame maximálná entropická odozva. Potom formálne definujeme maximálnu entropickú odozvu a nakoniec opisuje význam hodnôt maximálnej entropickej odozvy pre rôzne akcie u_t, ktoré sa vyberajú z diskrétnej množiny U. Definíciu, vlastnosti a opis maximálnej entropickej odozvy preberáme z článkov [13, 12]. Ostatné vysvetlenia, definície a optimalizačné problémy v tejto sekcií preberáme z rovnakých článkov [13, 12].

Podmienená pravdepodobnosť $p_t(u_t|u_{<t})$ meria budúcu flexibilitu agregátora (t. j. následujúce uskutočniteľné trajektórie akcií, ktoré sa uvoľnia vykonaním akcie u_t) na základe predchádzajúcich akcií $u_{<t} = (u_1, \ldots, u_{t-1})$ za podmienky, že si systémový operátor vyberie akciu u_t v čase t. Následujúce uskutočniteľné trajektórie akcií definujeme nižšie v tejto sekcii. Súčtom entropie podmienenej pravdepodobnosti p_t vyjadrujeme informatívnosť jej spätnej odozvy. Informatívnosť spätnej odozvy je dôležitá pre systémového operátora, lebo pomocou nej systémový operátor vie riešiť optimalizačný problém (1.11)-(1.14).

Množina všetkých uskutočniteľných trajektorií akcií (t. j. trajektorií akcií spĺňajúcich obmedzenia systému) je

$$S = \{u_{\leq T} \in U^T : u_{\leq T} \text{ spĺňa } (1.7) - (1.10)\}, \tag{2.1}$$

kde premenná $u_{\leq T}$ je $u_{\leq T} = (u_1, \ldots, u_T)$. Z dôvodov uvedených vyššie v tejto sekcií my vyberáme podmienenú distribúciu p_t pre $t = 1, \ldots, T$, ktorá maximalizuje svoju podmienenú entropiu. My uvažujeme stochasticky optimalizačný problém

$$F := \max_{p_1,\ldots,p_T} \sum_{t=1}^{T} H(\mathscr{U}_t|\mathscr{U}_{<t}) \tag{2.2}$$

vzhľadom na:

$$\mathscr{U} \in S, \tag{2.3}$$

kde premenné

$$p_t := p_t(\cdot,\cdot) := P_{\mathscr{U}_t|\mathscr{U}_{<t}}, t = 1, \ldots, T, \tag{2.4}$$

sú hustotné funkcie. Premenná $\mathscr{U} \in U^T$ je náhodná premenná. Premennú p_t označujeme ako podmienenú distribúciu v tomto optimalizačnom probléme z výpočtových dôvodov. Informácia poslaná systémovému operátorovi od agregátora je stále pravdepodobnostný vektor $p_t, t = 1, \ldots, T$ s dľžkou $|U|$.

Náhodná premenná $\mathscr{U}$ je distribuovaná na základe spoločnej distribúcie $\Pi_{t=1}^{T} p_t$. Entropia $H(\mathscr{U}_t|\mathscr{U}_{<t})$ je diskrétna entropia podmienenej distribúcie p_t

$$H(\mathscr{U}_t|\mathscr{U}_{<t}) := \sum_{u_{<t} \in U^t} (-\Pi_{l=1}^{t} p_l(u_l|u_{<l})) \cdot log p_t(u_t|u_{<t}). \tag{2.5}$$

Pomocou reťazového pravidla získame $\sum_{t=1}^{T} H(\mathscr{U}_t|\mathscr{U}_{<t}) = H(\mathscr{U})$. To znamená, že (2.2)-(2.5) môžeme chápať ako maximalizovanie entropie $H(\mathscr{U})$. Maximalizujeme entropiu $H(\mathscr{U})$ tak, že maximalizujeme entropiu spoločnej distribúcie $\Pi_{t=1}^{T} p_t$ z ktorej je množina (náhodná trajektória) $\mathscr{U} \in S$ vzorkovaná.

Optimalizačný problém (2.2)-(2.5) zahŕňa odozvu flexibility pre časový horizont $1, \ldots, T$. Jednotlivé premenné p_t sú podmienené pravdepodobnosti. Tieto jednotlivé podmienené pravdepodobnosti p_t sú zavísle len od informácií dostupných agregátorovi po čas t.

Definícia 2.1.1 *Ak sú hustotné funkcie $(p_1^*, \ldots, p_T^*)$ unikátnym riešením* (2.2)-(2.5)*, tak každá hustotná funkcia $p_t^* = \Psi_{SAC}^*(u_{<t})$, $t \in \mathscr{T}$ sa nazýva maximálna entropická odozva.*

Ak je daná akákoľvek trajektória akcií $u_{\leq t}$, tak množina následujúcich uskutočniteľných trajektorií je

$$S(u_{\leq t}) = \{v_{>t} \in U^{T-t} | \, v \text{ spĺňa } (1.8) - (1.10), v_{\leq t} = u_{\leq t}\}. \tag{2.6}$$

Teoréma 2.1.1 *Maximálna entropická odozva p_t^* v čase t je*

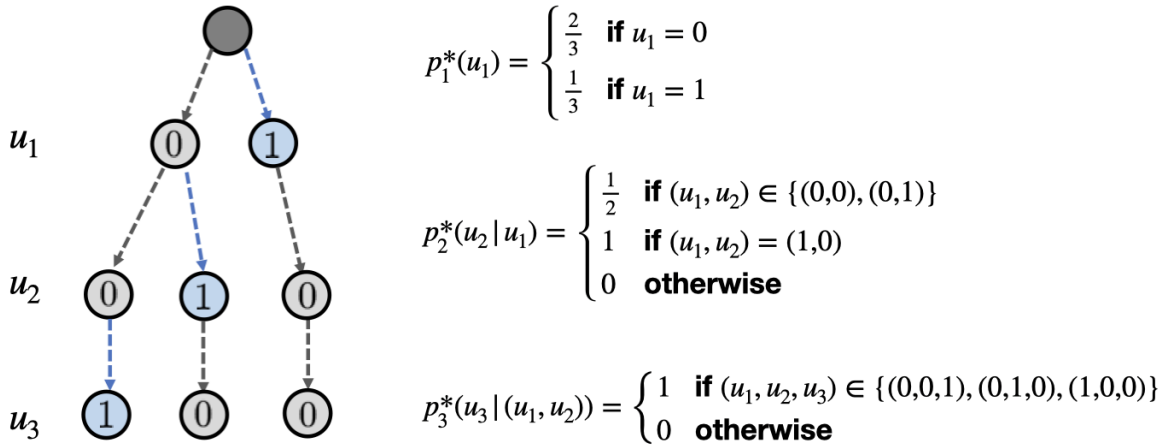
$$p_t^*(u_t|u_{<t}) = \frac{|S(u_{<t}, u)|}{|S(u_{<t})|}, \forall (u_{<t}, u_t) \in U^t, t \in \mathscr{T}. \tag{2.7}$$

Optimálna hodnota premmenej F v optimalizačnom probléme (2.2)-(2.5) *je $F = log|S|$.*

Maximálna entropická odozva p_t^* poskytuje správne informácie systémovému operátorovi, na to aby vedel vybrať vhodnú akciu u_t. Ak si systémový operátor vyberie akciu u_t s podmienenou pravdepodobnosťou $p_t^*(u_t) > 0$ pre každý čas t, tak potom trajektória akcií S bude uskutočniteľná $u \in S$ a systém zostáva uskutočniteľný po všetky časy $t \in \mathscr{T}$.

Ak si systémový operátor vyberie akciu u_t s väčšou maximálnou entropickou odozvou $p_t^*(u_t)$, tak potom bude systém viac flexibilný pre výber budúcich akcií ako keby si vybral signál u_t' s menšou maximálnou entropickou odozvou $p_t^*(u_t')$.

Vo zvyšku sekcie uvádzame príklad maximálnej entropickej odozvy v čase. V tomto príklade simulujeme nabíjanie jedného elektromobilu i s časom príchodu $a(i) = 1$, časom odchodu $d(i) = 3$, maximálnou rýchlosťou nabíjania $r(i) = 1\text{kW}$ a požadovanou energiou $e(i) = 1\text{kWh}$. Množina realizovateľných akcií (v tomto príklade v kWh) je $U = \{0,1\}$ a časový horizont je $\mathscr{T} = \{1,2,3\}$ a premenná Δ je jedna hodina. V každom čase $t \in \mathscr{T}$ elektromobil i sa môže dobíjať buď o 1kWh alebo o 0kWh.



Obr. 2.1: Príklad maximálnej entropickej odozvy. Zdroj obrázka: [12].

Pre nastavenia tohoto príkladu množina uskutočniteľných trajektórií je $S = \{(0,0,1),(0,1,0),(0,0,1)\}$, ktorú sme ukázali na obrázku 2.1. Optimálne podmienené distribúcie p_1^*, p_2^*, p_3^* sú tiež ilustrované na obrázku 2.1.

Z definície $H(\mathscr{U}_t|\mathscr{U}_{<t}) = H(\mathscr{U}_t) = H(p_t)$ vyplýva, že kvantita podmienená na premennú $u_{\leq t}$ je nepodmienená kvantita, a preto $p_t^*(u_t|u_{<t}) := p_t^*(u_t)$. Vynechávame premennú $u_{<t}$, kde to nie je nevyhnutné pre našu diskusiu.

2.2 Proces učenia

V tejto sekcii opisujeme proces učenia. Učenie prebieha v epizódach $(U_t, X_t, f_t)_{t=1}^T$, kde premenná T je dĺžka jednej epizódy. Pod jednou epizódou rozumieme jeden deň nabíjania. Trénovacie dáta každej epizódy sú historické vektory $(a(i), d(i), r(i), e(i))$. Tieto historické

vektory sú špecifikované používateľmi elektromobilov, ktorý navštívili nabíjaciu stanicu v príslušný deň (epizódu) [12].

2.2.1 Aproximovanie maximálnej entropickej odozvy

V tejto podsekcii opisujeme aký algoritmus učenia posilňovaním používame na aproximáciu maximálnej entropickej odozvy, aký optimalizačný problém algoritmus učenia posilňovaním rieši a opis generickej odmeňovacej funkcie pomocou ktorej dokáže algoritmus učenia posilňovaním aproximovať maximálnu entropickú odozvu. Popis algoritmu učenia posilňovaním, ktorý používame a priebeh učenia a opis generickej odmeňovacej funkcie sme prevzali z článkov [12, 25].

V praktických systémoch je presný výpočet maximálnej entropickej odozvy veľmi náročný. Preto my maximálnu entropickú odozvu aproximujeme pomocou algoritmu učenia posilňovaním založenom na aktér-kritik architektúre. Používame algoritmus hlbokého učenia posilňovaním SAC na aproximovanie maximálnej entropickej odozvy. Vykonávame off-policy učenie agregátorovej funkcie Ψ_{SAC}. Opis off-policy učenia je v článku [25]. Naučená agregátorova Ψ_{SAC} vracia aproximovanú maximálnu entropickú odozvu p_t. Cieľom algoritmu SAC je optimalizovať politiku/agregátorovu funkciu Ψ_{SAC}, tým že sa maximalizuje $J(\Psi_{SAC})$

$$J(\Psi_{SAC}) = \sum_{t=1}^{T} E_{(x_t,u_t,p_t)\sim p\Psi}[r(x_t,u_t,p_t) + \alpha H(\Psi_{SAC}(\cdot|x_t))], \tag{2.8}$$

kde p_Ψ sú marginálne rozdelenia stavov a akcií z distribúcie trajektórií, ktorá je vyvolaná politikou Ψ_{SAC}. Parameter teploty $\alpha > 0$ vyvažuje dôležitosť odmeny $r(x_t,u_t,p_t)$ a entropie $H(\Psi_{SAC}(\cdot|x_t))$. E je očakávaná hodnota náhodných premenných. Na úspešné aproximovanie maximálnej entropickej odozvy potrebujeme definovať vhodnú odmeňovaciu funckiu $r(x_t,u_t,p_t)$. My používame odmeňovaciu funkciu

$$r(x_t,u_t,p_t) = H(p_t) + \sigma h(x_t,u_t,X_t,U_t), \tag{2.9}$$

kde je dôležitý výraz $H(p_t)$, lebo maximalizuje entropiu hustotnej funkcie p_t na základe definície maximálnej entropickej odozvy. Premenná σ je škála odmeny. Výraz $\sigma h(x_t,u_t,X_t,U_t)$ odmeňuje stav $x_t \in X_t$ a akciu $u_t \in U_t$ v prípade, keď sú splnené obmedzenia systému (1.4) a (1.5). Zostávajúce obmedzenia (1.2), (1.3), (1.6) sú vždy splnené zvolením vhodného jednoduchého plánovacieho algoritmu.

Akcie a stavy, ktoré porušujú obmedzenia (1.4) a (1.5) sa trestajú. Odmeňovacia funkcia $r(x_t,u_t,p_t)$ je nezávislá od cenových funkcií. Aproximovaná maximálnej entropická odozva môže byť odlišná od maximálnej entropickej odozvy v definovanej v sekcii 2.1. Napríklad ak si systémový operátor vyberie akciu u_t s podmienenou pravdepodobnosťou $p_t^*(u_t) > 0$, pre každý čas t tak nevieme zaručiť, že nebudú porušené obmedzenia systému, t. j. $u \in S$.

Maximálna entropická odozva p_t musí byť definovaná v pravdepodobnostnom simplexe P

$$P := \{p \in R^{|U|} : p(u) \geq 0, u \in U, \sum_{u \in U} p(u) = 1\}. \tag{2.10}$$

2.2.2 Odmeňovacia funkcia

V tejto podsekcii opisujeme odmeňovaciu funkciu, ktorú sme použili v našom riešení. Pri návrhu a opise odmeňovacej funkcie sme sa inšpirovali článkami [12, 11]. Prepokladáme, že odmeňovacia funkcia je nezávislá od cenových funkcií. Tvar odmeňovacej funkcie (ako špecifický príklad generickej odmeňovacej funkcie (2.9)), ktorú používame je

$$\begin{gathered} r(\text{Stav agregátora v čase } t = x_t, \\ \text{Signál, ktorý systémový operátor poslal agregátorovi v čase } t = u_t, \\ \text{Spätná odozva o flexibilite generovaná agregátorom v čase t} = p_t) = \\ = \text{entropia } p_t \\ + o_1 \cdot \sum_{i \in N'} ||s_t(i)||_2 \\ - o_2 \cdot \sum_{i \in N'} (I(0 < d_t(i) \leq \Delta) \cdot (e_t(i) - s_t(i)) \\ - o_3 \cdot |u_t - \sum_{i \in N'} s_t(i)| \end{gathered}$$

kde plán nabíjania s_t generujeme na základe signálu u_t a $I(\cdot)$ je indikátorova funkcia a N' je množina nabíjaných elektromobilov v čase t. Výrazom $||s_t(i)||_2$ rozumieme l_2-normu výrazu $s_t(i)$. Pre daný vektor x je l_2-norma vektora x (definovaná v dokumente [24]) $||x||_2 = (|x_1|^p + \cdots + |x_n|^p)^{\frac{1}{p}}$.

Popis jednotlivých prvkov v odmene $r(x_t, u_t, p_t)$:

1. výraz je pažravá aproximácia definície maximálnej entropickej odozvy (maximálna entropická odozva je definovaná v sekcii 2.1). Pomocou tohoto výrazu dokážeme maximalizovať entropiu spätnej odozvy o flexibilite p_t.

2. výraz pomáha zväčšiť množstvo dodanej energie.

3. výraz zabezpečuje väčšie dodržiavanie obmedzenia (1.5).Nedodanie požadovaného množstva energie do času odchodu je potrestané (t. j. stav a akcia sa trestajú).

4. výraz zabezpečuje väčšie dodržiavanie obmedzenia (1.4).

Politika konverguje pre širokú škálu parametrov o_1, o_2, o_3. Predpokladáme, že $o_1, o_2, o_3 > 0$.

2.3 Online penalizované prediktívné riadenie

V tejto sekcii opisujeme algoritmus online penalizovaného prediktívneho riadenia, pomocou ktorého systémový operátor dokáže riešiť optimalizačný problém (1.7)-(1.10). Pri vývoji a opise algoritmu penalizovaného prediktívného riadenia sme sa inšpirovali publikáciami [12, 11].

Algoritmus online penalizovaného predikívneho riadenia (PPC) dizajnuje systémový operátor. Algoritmus PPC je metóda založená na modelovom prediktívnom riadení (MPC) s penalizačným prvkom (aproximovanej) maximálnej entropickej odozvy p_t^* v riešenom online optimalizačnom probléme

$$u_t = \underset{u_t \in U}{arg\,inf}\,(c_t(u_t) - \beta log\, p_t^*(u_t|u_{<t})), \tag{2.11}$$

ktorý je ekvivalentný online verzii optimalizačnému problému (1.7)-(1.10) pre $\beta > 0$. Ak $\beta = 0$, tak riešením optimalizačného problému je signál s najnižšou cenou, t. j. $u_t = 0$.

```
1 min_{u_t} = ∞
2 for u_t in U do
3       if p_t^*(u_t|u_{<t}) > 0 and (c_t(u_t) − β log p_t^*(u_t|u_{<t})) < min_{u_t} do
4           min_{u_t} = u_t
5 return min_{u_t}
```

Obr. 2.2: Pseudokód online algoritmu PPC.

Ak je maximálna entropická odozva $p_t^*(u_t|u_{<t}) = 0$, tak potom je výraz $log_e p_t(u_t|u_{<t})$ nedefinovaný. Preto hodnoty u_t pre ktoré platí $p_t(u_t|u_{<t}) = 0$, nie sú nikdy riešením optimalizačného problému (2.11).

Akcia u_t v optimalizačnom probléme (2.11), ktorá zabezpečuje najvyššiu spätnú odozvu o flexibilite $p_t^*(u_t|u_{<t})$ nemusí byť tá, ktorá vie minimalizovať cenu c_t za nakúpenú energiu u_t. Platí to aj pre opačný prípad. Parameter β používame na to, aby sme vybalancovali tieto dva ciele a našli signál u_t, ktorý minimalizuje kumulatívne náklady a zabezpečuje dodržanie obmedzení systému. Čím je vyvažovací parameter β vyšší, tým vieme lepšie dodržať obmedzenia systému, ale za cenu vyšších nákladov za nakúpenú energiu.
Ak systémový operátor chce limitovať mieru sklonu tak zavedie operačné obmedzenie

$$|u_{t+1} - u_t| \leq \varepsilon, t = 1, \ldots, T, \tag{2.12}$$

kde $\varepsilon \in R^+$ je limit miery sklonu. Systémový operátor dokáže aj limitovať najvyššiu odkúpenú energiu v čase

$$u_t \leq \gamma, t = 1, \ldots, T, \tag{2.13}$$

kde $\gamma \in R^+$ je limit na najvyššiu odkúpenú energiu v čase.

2.4 Systém uzavretej slučky

V tejto subsekcii opisujeme systém uzavretej slučky, ktorý používame na riešenie optimizačného problému (1.11)-(1.14). Pri dizajnovaní a opise systému uzavretej slučky sme sa inšpirovali článkami [11, 12].

Systém uzavretej slučky je každý systém, ktorý dokáže reagovat na zmeny a samostatne spraviť korekcie chýb na základe spätnej odozvy [26]. Na začiatku systému uzavretej slučky v každom čase $t \in \{1,\ldots,T\}$ generuje systémový operátor signál $u_t \in U$ pomocou algoritmu PPC (opísaný v sekcií 2.3). Signál u_t vygeneruje systémový operátor základe pravdepodobnostného vektora $p_t \in P$ s dĺžkou $|U|$ a cenovej funkcie c_t.

Cenu za kúpu energie u_t si zahrnie systémový operátor v svojich kumulatívnych nákladoch C_t. Potom agregátor zmení svoj stav x_t v čase t na stav x_{t+1}. Zmena stavu pozostáva z viacerých krokov:

1. Agregátor zaznamená vo svojom stave x_t príchod nových elektromobilov v čase $t+1$. Takýmto elektromobilom pridelí voľné nabíjačky. V nami definovanom modeli systému predpokladáme, že vždy dostatok voľných nabíjačok pre prichádzajúce elektromobily.

2. Agregátor vygeneruje plán nabíjania s_t pre čas t pomocou plánovacieho algoritmu π a na základe signálu u_t. Každému aktívnem (pripojenému) elektromobilu i dodá $s_t(i) \in R^+$ energie. Agregátor zmení svoj stav z x_t na x_{t+1} pomocou deterministickej prechodovej funkcie f_t, ktorá aktualizuje stav každého nabíjaného elektromobilu $i \in N'$

$$e_t(i) = e_{t-1}(i) - s_t(i),$$
$$d_t(i) = d_{t-1}(i) - \Delta,$$

   kde $e_t(i)$ je zostávajúce množstvo požadovanej energie elektromobilu i v čase t, $d_t(i)$ je zostávajúci čas nabíjania (v hodinách) elektromobilu i v čase t.

3. Agregátor zaznamená vo svojom stave x_{t+1} odchod elektromobilov v čase t. Každý elektromobily i odíde z nabíjacej stanice v čase $t+1$, ak $d_t(i) - \Delta = d_{t+1}(i) = 0$.

Stav x_t sa v implementácii odlišuje od stavu x_t opísaného v sekcii 1.1 tým, že stav agregátora x_t je vektor stavov všetkých nabíjačiek. Ak na nabíjačku nie je pripojený žiaden elektromobil, tak stav nabíjačky je $(0,0)$. Ak je na nabíjačku pripojený elektromobil i v čase t, tak jej stav je $(d_t(i), e_t(i))$.

V poslednom kroku systému v čase t agregátor sa učí alebo generuje flexibilitu p_{t+1} na základe predchádzajúcich signálov $u_{<t}$. Potom agregátor posiela spätnú odozvu o flexibilite p_{t+1} systémovému operátorovi a cyklus sa opakuje.

```
1  p_1 = [0]^|U|
2  p_1^(1) = 1
3  x_1^(k) = (0,0), k ∈ K
4  K_v = K
5  for t in 1,...,T do
6        u_t = PPC(p_t, c_t)
7        C_t = C_{t-1} + c_t u_t
8        for i in N do
9              if t < a(i) ≤ t+1 and ∃k ∈ K_v then
10                   x_t^(k) = (d(i) − t, e(i))
11                   K_v = K_v \ {k}
12       s_t = π(x_t, u_t)
13       K_o = K \ K_v
14       for k in K_o do
15             if 0 < x_t^(k)(1) ≤ Δ do
16                   x_{t+1}^(k) = (0,0)
17                   K_v = K_v ∪ {i}
18             else
19                   x_{t+1}^(i)(1) = x_t^(k)(1) − Δ
20                   x_{t+1}^(i)(2) = x_t^(k)(2) − s_t(i)
21
22       p_{t+1} = Ψ_SAC(x_{t+1})
23 return C_t
```

Obr. 2.3: Pseudokód systému uzavretej slučky. Premenná K_v vyjadruje voľné nabíjačky a premenná K_o vyjadruje obsadené nabíjačky. Tento systém sme dizajnovali na základe článkov [11, 12].

Predpokladáme, že v systéme uzavretej slučky v každom čase t agregátor vie len o stavoch $(x_1,\ldots,x_t)$, prechodových funkciách $(f_1,\ldots,f_T)$ a signáloch $(u_1,\ldots,u_t)$. Systémový operátor v každom čase t vie len o cenových funkciách $(c_1,\ldots,c_t)$ a spätných odozvách flexibility $(p_1,\ldots,p_t)$.

2.5 Proces testovania

S naučenou agregátorovou funkciou $\Psi_{SAC} : X \to P$, ktorá na základe stavu agregátora generuje aproximáciu maximálnej entropickej odozvy (ktorá sa odlišuje od definovanej maximálnej entropickej odozvy definovanej v 2.1) vieme aplikovať systém uzavretej slučky

v reálnom čase na riešenie optimalizačného problému (1.11)-(1.14). Ide o totožný systém uzavretej slučky ako v sekcii 2.4. Agregátorova funkcia Ψ_{SAC} je parametrizovaná hlbokou neurónovou sieťou.

2.6 Jednoduché online plánovacie algoritmy

V tejto sekcii sa opisujeme vybrané jednoduché online plánovacie algoritmy. Jednoduché online plánovacie algoritmy least-laxity-first a earliest-deadline-first opisujeme v podsekcii 2.6.1. Jednoduchý online plánovací algoritmus smoothed least-laxity-first opisujeme v podsekcii 2.6.2. Každý online plánovací algoritmus musí zabezpečiť platnosť obmedzení: (1.2),(1.3),(1.6). Jednoduché plánovacie algoritmy označujeme grécky písmenom π. Vstupom do každého plánovacieho algoritmu π je stav agregátora x_t a energia u_t v čase t. Výstupom plánovacieho algoritmu je plán nabíjania s_t elektromobilov pre čas t [12].

2.6.1 Least-Laxity-First

V tejto podsekcii opisujeme jednoduché online algoritmy least-laxity-first a earliest-deadline-first. Opis algoritmov least-laxity-first a earliest-deadline-first čerpáme z publikácii [17, 14, 27].

Algoritmus least-laxity-first (LLF) je online plánovací algoritmus. Hlavným cieľom algoritmov least-laxity-first a earliest-deadline-first je maximalizovať množstvo nabitej energie. Kroky algoritmu LLF sú nasledovné:

1. zoradenie pripojených (aktívnych) elektromobilov podľa laxity vo vzostupnom poradí

2. spracovávať nabíjanie elektromobilov v takomto poradí

3. každému pripojenému elektromobilu $i \in N'$ je pridelená maximálna uskutočniteľná rýchlosť nabíjania $s_t(i)$ v čase t. Maximálnu uskutočniteľnú rýchlosť nabíjania $s_t(i)$ vypočítame pomocou bisekcie. Postupne v danom poradí agregátor dodáva dostupnú energiu u_t elektromobilom.

Výpočet laxity sa nachádza v kapitole 4. V tomto kontexte je maximálna uskutočniteľná rýchlosť nabíjania $s_t(i)$ taká, ktorá neporušuje obmedzenie

$$\sum_{i \in N} s_t(i) \leq u_t, t = 1, \ldots, T, \tag{2.14}$$

a je nižšia ako maximálna rýchlosť nabíjania $r(i)$ v algoritme LLF (pozri obmedzenie (1.6)).

```
1  P_t = u_t
2  sorted_active_evs = sort(N', key=laxity)
3  for i in sorted_active_evs do
4      s_t(i) = bisection(r(i), P_t, e_t(i))
5      P_t = P_t − s_t(i)
6  return s_t
```

Obr. 2.4: Pseudokód algoritmu LLF pre jeden časový krok $t \in \mathscr{T}$.

Premenná sorted_active_evs na obrázku 2.4 je množina pripojených elektromobilov v čase t usporiadaná od elektromobilov s najnižšou laxitou po elektromobily s najvyššou laxitou v čase t.

Tolerovateľná chyba bisekčnej metódy je $\varepsilon = 1 \cdot 10^{-4}$. Bisekčnú metódu vykonáva agregátor pre každý elektromobil zvlášť. Dolná hranica optimalizovanej premennej $s_t(i)$ je 0. Horná hranica optimalizovanej premennej $s_t(i)$ je $min(r(i), e_t(i))$ pre elektromobil $i \in N'$. Na obrázku 2.5 je lb dolná hranica optimalizovanej premennej a ub horná hranica optimalizovanej premennej. Bisekčná metóda vracia je maximálna realizovateľná rýchlosť nabíjania $s_t(i)$, ktorá je striktne nižšia ako maximálna rýchlosť nabíjania $r(i)$ a neporušuje obmedzenia infraštruktúry.

```
1  ε = 1 · 10^−4
2  lb = 0
3  ub = min(r(i), e_t(i))
4  while |ub − lb| > ε do
5      s_t(i) = (ub + lb)/2
6      if s_t(i) > P_t then
7          ub = s_t(i)
8      else
9          lb = s_t(i)
10 s_t(i) = lb
11 return s_t(i)
```

Obr. 2.5: Pseudokód bisekčnej metódy používanej v algoritme LLF. Bisekčnú metódu sme navrhli na základe inšpirácie z článku [28].

Algoritmus earliest-deadline-first funguje na rovnakom princípe ako least-laxity-first, s tým rozdielom, že algoritmus earliest-deadline-first triedi elektromobily v kroku 1 od elektromobilov s najskoršími odchodmi po elektromobily s najneskoršími odchodmi.

2.6.2 Smoothed Least-Laxity-First

V tejto podsekcii opisujeme jednoduchý plánovací algoritmus smoothed least-laxity-first. Opis algoritmu smoothed least-laxity-first (sLLF) sme prevzali z publikácií [14, 27].

Algoritmus smoothed least-laxity-first je online algoritmus, ktorý je vylepšením algoritmu least-laxity-first. Vylepšenie smoothed least-laxity-first oproti klasickému least-laxity-first spočíva v tom, že smoothed least-laxity-first nemá nepotrebné oscilácie/výkyvy v rýchlosti nabíjania elektromobilov. Oscilácie v rýchlosti nabíjania elektromobilov možu skrátiť životnosť niektorých batérii elektromobilov (napríklad batéria Li-on).

Prístup použitý v algoritme sLLF sa zameriava na maximalizáciu minimálnej laxity spomedzi všetkých elektromobilov t. j. $max_{s_{1:T}} min_{i \in N} l_T(i)$. Premenných $s_{1:T} = \{s_t(i) | \, i \in N, t = 1, \ldots, T\}$, ktoré sú riešením optimalizačného problému $max_{s_{1:T}} min_{i \in N} l_T(i)$ môže byť viacero. Vyriešením následujúceho offline optimalizačného problému dokážeme nájsť unikátne riešenie $s_{1:T}$ také, ktoré je aj riešením optimalizačného problému $max_{s_{1:T}} min_{i \in N} l_T(i)$

$$s_{1:T} = \underset{s_{1:T}}{argmax} \sum_{i \in N} r(i) \cdot q(l_T(i)) \tag{2.15}$$

vzhľadom na:

$$(1.2), (1.3), (1.6),$$

$$\sum_{i \in N} s_t(i) \leq u_t, t = 1, \ldots, T, \tag{2.16}$$

$$\sum_{t=1}^{T} s_t(i) \leq e(i), i \in N. \tag{2.17}$$

Kde q je dvojito diferencovateľná, striktne konkávna a monotónne rastúca funkcia. Premenná $l_T(i)$ je laxita elektromobilu i v čase T a premenná $r(i)$ je maximálna rýchlosť nabíjania elektromobilu i.

Jednoduchý plánovací algoritmus sLLF je online algoritmus, a preto nedisponuje informáciami o príchodoch budúcich elektromobilov. Z toho dôvodu je potrebné aproximovať riešenie offline optimalizačného problému (2.15)-(2.17) pomocou online optimalizačného problému. Tvar online optimalizačného problému, ktorý rieši online algoritmus sLLF v každom čase $t \in \mathscr{T}$ je

$$\max_{s_t} \sum_{i \in N'} r(i) \cdot q(l_{t+1}(i)) \tag{2.18}$$

vzhľadom na:

$$(1.2), (1.3), (1.6),$$

$$\sum_{i \in N} s_t(i) \leq u_t, t = 1, \ldots, T, \tag{2.19}$$

$$s_t(i) \leq e_t(i), i \in N'. \tag{2.20}$$

Kde $e_t(i)$ je zostávajúce množstvo požadovanej energie pre elektromobil i v čase t. Optimalizačný problém (2.18)-(2.20) maximalizuje minimálnu laxitu pre budúci časový krok $t+1$,

t. j. $max_{s_t} min_{i \in N'} l_{t+1}(i)$. Riešením optimalizačného problému (2.18)-(2.19) je

$$s_t^*(i) = [r(i) \cdot (L_t - l_t(i) + 1)]_0^{min(e_t(i), r(i))}, i \in N'. \tag{2.21}$$

Výraz $[x]_a^b$ je projekcia skalára na interval $[a,b]$. To znamená, že ak $x > b$ tak hodnota projekcie x na interval $[a,b]$ bude $x = b$ a ak $x < a$ tak analogicky $x = a$. Premenná s_t^* je výsledný plán nabíjania s_t

$$\sum_{i \in N'} s_t^*(i) = min(u_t, \sum_{i \in N'} min(r(i), e_t(i))). \tag{2.22}$$

Na nájdenie optimálnej hodnoty L_t agregátor využíva binárne vyhľadávanie (bisekciu). Po zistení hodnoty L_t agregátor vypočíta plán nabíjania s_t všetkých aktívnych elektromobilov.

$L_t = bisection(N', u_t)$
for i in N' do
    $s_t(i) = [r(i) \cdot (L_t - l_t(i) + 1)]_0^{min(e_t(i), r(i))}$
return s_t

Obr. 2.6: Pseudokód algoritmu sLLF pre jeden časový krok $t \in \mathscr{T}$.

Počiatočnú hodnotu dolnej hranice v bisekcii (premenná *lb* na obrázku 2.7) je hodnota premennej L_t, ktoré zabezpečí, že sa nebude nabíjať žiadny elektromobil. Počiatočnú hodnotu hornej hranice v bisekcii (premenná *ub* na obrázku 2.7) je hodnota premmenej L_t, ktorá zabezpečí, že sa všetky aktívne elektromobili budú nabíjať čo najviac energie. Následne bisekčná metóda hľadá optimálnu L_t pre ktorú s_t^* neporušuje obmedzenia infraštruktúry. Bisekčná metóda na konci vráti aktualizovanú dolnú hranicu L_t ako optimálnu L_t, lebo je overené pomocou bisekcie, že spĺňa obmedzenia infraštruktúry. Premenná ε je tolerovateľná chyba a nastavujeme ju na hodnotu $1 \cdot 10^{-4}$.

$\varepsilon = 1 \cdot 10^{-4}$
$ub = max(\{l_t(i) | i \in N'\})$
$lb = min(\{l_t(i) | i \in N'\}) - 1$
while $|ub - lb| > \varepsilon$ do
    $L_t = (ub - lb)/2$
    if $\sum_{i \in N'} [r(i) \cdot (L_t - l_t(i) + 1)]_0^{min(e_t(i), r(i))} > u_t$ do
        $ub = L_t$
    else
        $lb = L_t$
$L_t = lb$
return L_t

Obr. 2.7: Kód bisekčnej metódy používanej v algoritme sLLF. Bisekčnú metódu sme navrhli na základe inšpirácie z článku [28].

2.7 Offline optimálny algoritmus

V tejto sekcii predstavujeme offline optimálny algoritmus, aký optimalizačný problém rieši a za akých predpokladov (inšpirovali sme sa opismi offline optimálneho algoritmu v článkoch [12, 14, 15]). Offline optimálny algoritmus disponuje informáciami o príchodoch budúcich elektromobiloch, vrátane ich požadovanej energie, času odchodu a maximálnej rýchlosti nabíjania. Budúce ceny za nabíjanie musia byť offline optimálnemu algoritmu známe, aby vyriešil offline optimalizačný problém. Offline optimálny algoritmus rieši optimalizačný problém

$$s_{1:T} = \sum_{t=1}^{T} c_t(\sum_{i \in N} s_t(i)) \tag{2.23}$$

vzhľadom na:

$$s_t(i) = 0, t < a(i), i \in N, t = 1, \ldots, T, \tag{2.24}$$

$$s_t(i) = 0, t > d(i), i \in N, t = 1, \ldots, T, \tag{2.25}$$

$$\sum_{i \in N} s_t(i) = u_t, t = 1, \ldots, T, \tag{2.26}$$

$$\sum_{t=1}^{T} s_t(i) = e(i), i \in N, \tag{2.27}$$

$$0 \leq s_t(i) \leq r(i), t = 1, \ldots, T, \tag{2.28}$$

kde optimalizovaná premenná $s_{1:T} = (s_1, \ldots, s_T)$ je plán nabíjania všetkých elektromobilov pre (optimalizačný) časový horizont $t = 1, \ldots, T$. Optimalizačný problém (2.23)-(2.28) je rovnaký ako optimalizačný problém (1.11)-(1.14).

Agregátor rieši optimalizačný problém (2.23)-(2.28) pomocou offline optimálného algoritmu. Predpokladáme, že riešenie optimalizačného problému (2.23)-(2.28) nezahŕňa koordináciu agregátora a systémového operátora.

3 Dáta

Používame skutočné dáta nabíjaní elektrických vozidlách z dátovej množiny ACN-Data. Dátová množina ACN-Data pozostáva z viac než 66 000 záznamov o nabíjaní elektrických vozidiel, ktoré boli získané z troch nabíjacích sietí v Californii. Tieto tri nabíjacie siete sa nazývajú ACN (má 54 nabíjačiek), JPL (má 52 nabíjačiek) a Office 1 (má 8 nabíjačiek) [29, 30]. Záznamy o nabíjaní elektromobilov sú denne pridávané do dátovej množiny ACN-Data. Dátová množina ACN-Data je verejne dostupná na stránke uvedenej v zdroji [31]. (viac informácii o dátovej množine ACN-Data sa nachádza v článku [29]). My pristupujeme k dátovej množine ACN-Data programátorsky cez jednoduché klientské rozhranie pre používanie API v programovacom jazyku Python. Klientovské rozhranie, ktoré používame, je súčasťou balíka acnportal [32].

3.1 Forma dát


```
{
"_id": "5bc90cb9f9af8b0d7fe77cd6",
"userInputs": null,
"sessionID": "2_39_79_381_2018-04-25 14:40:33.638896",
"stationID": "2-39-79-381",
"spaceID": "CA-490",
"siteID": "0002",
"clusterID": "0039",
"connectionTime": "Wed, 25 Apr 2018 14:40:34 GMT",
"disconnectTime": "Wed, 25 Apr 2018 23:03:12 GMT",
"kWhDelivered": 10.119,
"doneChargingTime": "Wed, 25 Apr 2018 17:40:30 GMT",
"timezone": "America/Los_Angeles",
"userID": null
}
```


Obr. 3.1: Príklad záznamu nabíjania elektrického vozidla pripojeného na nabíjaciu stanicu 25.apríla 2018 získaný z webového rozhrania stránky [31]. Časy (Vstupné dáta) connectionTime a disconnectTime konvertujeme do časovej zóny v Los Angeles.

Parameter	Vysvetlenie
connectionTime	Čas, keď sa elektromobil pripojí na nabíjačku.
disconnectTime	Čas, keď sa elektromobil odpojí od nabíjačky.
doneChargingTime	Čas posledného dobitia elektromobilu.
siteID	Identifikátor nabíjacej siete, kde prebehlo nabíjanie.
stationID	Unikátny identifikátor nabíjačky, na ktorú je elektromobil napojený.
kWhDelivered	Zaznamenaná dodaná energia.
timezone	Časová zóna nabíjacej siete.
userID	Unikátny identifikátor vodiča elektromobilu.

Tabuľka 3.1: Opis parametrov z dátovej množiny ACN-Data. Tabuľku sme prevzali z [29].

Tabuľka 3.1 opisuje parametre, ktoré sú uvedené v zázname nabíjania zobrazenom na obrázku 3.1. Záznam nabíjania môže obsahovať aj ďalšie nepovinné parametre, ktoré nie sú uvedené zázname nabíjania 3.1. Parameter kWhRequested (množstvo požadovanej energie) je nepovinný parameter, a preto v experimentoch nastavujeme kWhDelivered v zázname nabíjania elektromobilu i ako množstvo požadovanej energie pre elektromobil i. Parameter, ktorý špecifikuje maximálnu rýchlosť nabíjania nie je uvedený vo viacerých záznamoch nabíjania elektromobilov. Z toho dôvodu (aj z inšpirácie z článku [13]) my nastavujeme maximálnu rýchlosť nabíjania elektromobilov na rovnakú hodnotu uvedenú v experimentoch [29].

4 Experimenty

V tejto kapitole opisujeme knižnice, nastavenia a implementácie, ktoré sme použili (prevzali alebo sami implementovali) na riešenie problému optimalizácie nabíjania. Opisujeme aj metriku na meranie chýb vo výslednom riešení. Prezentujeme výsledky experimentov a diskutujeme o týchto výsledkoch.

Agentov trénujeme na dátach o nabitých elektromobiloch z nabíjacích sietí Caltechu a JPL od 1. Novembra 2018 do 1. Decembra 2019. Víkendy a dni nabíjania s menej ako 30 elektromobilmi sme odstránili z trénovacích dát. Agentov testujeme na dátach z nabíjacej sietí Caltech a JPL od 2. Decembra 2019 do 1. Januára 2020. Víkendy a dni nabíjania s menej ako 30 elektromobilmi sme odstránili z trénovacích a testovacích dát. Vďaka odstráneniu daných dní zabezpečíme, že nie je posun medzi distribúciami trénovacích dát a testovacích dát. Ak existuje taký posun medzi distribúciami trénovacích dát a testovacích dát, tak algoritmy učenia posilňovaním, ktoré z týchto dát čerpajú, môžu mať horší výkon. Elektromobily, ktoré nabíjajú viacero dní sú tiež odstránené z trénovacej a testovacej množiny, lebo platí, že $1 \leq a(i) \leq T$ a $1 \leq d(i) \leq T$ [12, 11, 33].

Implementovali sme použité algoritmy v programovacom jazyku Python. Použili sme cvxpy solver SCIP na riešenie optimalizačného problému pomocou offline optimálneho algoritmu [34, 35]. Algoritmus SAC sme prebrali z balíka Stable-Baselines3, ktorý pozostáva z implementácii algoritmov učenia posilňovaním v pythonovskej knižnici Pytorch [36, 37]. Cenová funkcia použitá v procese trénovania je lineárna funkcia $c_t = 1 - t/24$, kde t je časový krok konvertovaný do hodín. Koeficienty odmeňovacej funkcie o_1, o_2, o_3 nastavujeme na $o_1 = 0.1$, $o_2 = 0.2$ a $o_3 = 2$. Cenovú funkcia v procese testovania uvádzame v experimentoch v tejto kapitole [12].

Agentov trénujeme a testujeme cez nami naprogramované prostredie pomocou knižnice Gymnasium [38]. Algoritmy na trénovanie a testovanie prostredia sme prevzali z dokumentácií Stable-Baselines3 [37] a Gymnasium [38]. Politika a odmeny konvergujú pre natrénovaných agentov, ak neuvádzame inak. Počas trénovania agentov si vyberáme nabíjacie dni (trénovacie epizódy) z vybraných nabíjacích sietí náhodne.

Pre jednoduchosť a na základe inšpirácie z článku [15] sme pri riešení optimalizačného problému 1.7-1.10 pomocou offline optimálneho algoritmu nastavili množinu realizovateľných akcii $U = R$. Algoritmus PPC pracuje s diskrétnou množinou U v experimentoch.

Dĺžka vektora p_t v našej implementácii je $|U_t|$, kde veľkosť množiny U_t je rovnaká pre

všetky $t \in \mathscr{T}$, ak neuvádzame inak. Technicky je možné zmeniť dĺžku vektora p_t na $|U|$, tým že nastavíme spätnú odozvu o flexibilite $p_t(u_t|u_{<t}) = 0$ pre akcie u_t, ktoré nespľňajú operačné obmedzenia, t.j. $u_t \notin U_t$. My neuvádzame veľkosť množiny U a predpokladáme, že je to diskrétna množina akcií.

4.1 Metrika merania porušení obmedzení systému

V tejto sekcii opisujeme metriku merania porušovania chýb (1.4)-(1.5), ktorú používame. Opis a matematické vyjadrenia chýb, ktorými meriame porušenia obmedzení systému (1.4)-(1.5) sme prevzali z článku [12].

Mieru porušenia obmedzenia (1.4) meriame normalizovanou priemernou kvadratickou chybou nazývanou MSE

$$MSE := \sum_{k=1}^{L} \sum_{t=1}^{T} |\sum_{i \in N} s_t^{(k)}(i) - u_t^{(k)}|^2 / (LxTx\gamma),$$

kde L je počet testovacích epizód, T je dĺžka jednej testovacej epizódy a N je množina všetkých obsluhovaných elektromobilov v daných testovacích epizódach. Premenná $u_t^{(k)}$ je signál v čase t a v episóde k a $s_t^{(k)}(i)$ je plán nabíjania elektromobilu i v čase t a v epizóde k. Operačné obmedzenie zavedené systémovým operátorom je $u_t \leq \gamma$.
Porušovania obmedzenia (1.5) meriame priemernou percentuálnou chybou nazývanou MPE

$$MPE := 1 - \sum_{k=1}^{L} \sum_{t=1}^{T} \sum_{i \in N} s_t^{(k)}(i) / (LxTx \sum_{i \in N} e(i)),$$

kde platí, že $0 \leq MPE \leq 1$.

4.2 Hyperparametre v experimentoch

Hyperparametre, ktoré nastavujeme pred začatím procesov učenia a testovania agentov preberáme z článkov [12] a [25]. List hyperparametrov, ktoré nastavujeme v experimentoch sú z článkov [12, 25]. Hyperparemetre/nastavenia, ktoré nemeníme v experimentoch sú

1. Algoritmus učenia posilňovaním = SAC
2. Optimizátor = Adam
3. Rýchlosť učenia = $3 \cdot 10^{-4}$
4. Veľkosť replay buffera = 10^6
5. Počet skrytých vrstiev (pre všetky neurónové siete) = 2
6. Množstvo neurónov pre každú skrytú vrstvu = 256

7. Množstvo vzoriek pre každú minidávku = 256
8. Aktivačná funkcia na zabezpečenie nelinearity = ReLU
9. Zľavový faktor = 0.5
10. Cieľový vyhladzovací koeficient (τ) = $5 \cdot 10^{-3}$
11. Kroky gradientu = 1
12. Cieľový interval aktualizácie = 1

Cieľový vyhladzovací koeficient (τ) je možné optimalizovať na zrýchlenie učenia. My premennú τ neoptimalizujeme a nastavujeme τ fixne na hodnotu $\tau = 5 \cdot 10^{-3}$ [25].

List zvyšných hyperparametrov, ktoré nastavujeme (a opisujeme/definujeme) na základe článkov [11, 12, 13] v nami implementovaných metódach a v nami implementovanom prostredí sú:

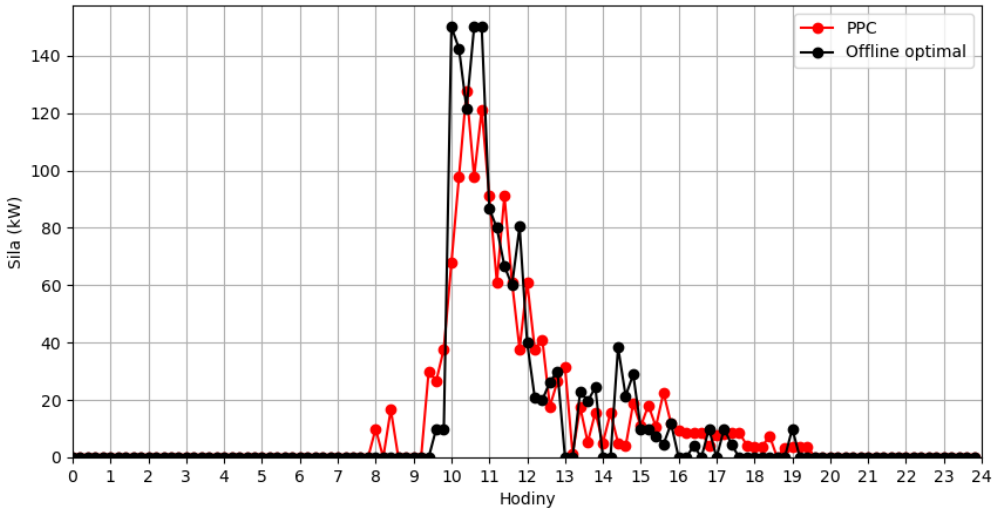
1. Vyhladzovací parameter $\beta \in [1 \cdot 10^3, 1 \cdot 10^6]$
2. Limit na maximálnu energiu $u_t <$ 150kW, to znamená, že $\gamma = 150$.
3. Parameter teploty (α) = 0.5
4. Plánovacie algoritmy = LLF/sLLF
5. Laxita elektromobilu i v čase t je $l_t(i) = d_t(i) - \frac{e_t(i)}{r(i)}$
6. Maximálna rýchlosť nabíjania pre každý elektromobil ($r(i)$) = 10kW
7. Časový interval Δ = 12 minút = 0.1 hodín
8. Akčný priestor prostredia (je to priestor v ktorom je výstupná spätná odozva o flexibilite p_t z neurónových sietí) = $[0,1]^{|U_t|}$. Výstupy z neurónových sietí (t. j. z funkcie Ψ_{SAC}) normalizujeme, aby bola učiaca sa/naučená maximálna entropická odozva p_t definovaná v priestore všetkých možných maximálnych entropických odoziev P.
9. Stavový priestor učenia posilňovaním X (založená na stave agregátora x_t). Stavový priestor učenia posilňovaním je vektor s dimenziou $|K| * 2$, kde K je množina všetkých nabíjačiek v nabíjacej stanici.

Stavový priestor učenia posilňovaním X je štandardne definovaný v učení posilňovaním a odlišuje sa od stavového priestoru v sekcii X definovanej v sekcii 1.1. Akčný priestor je tiež štandardne definovaný v učení posilňovaním a je odlišný od množiny realizovateľných akcií U [12].

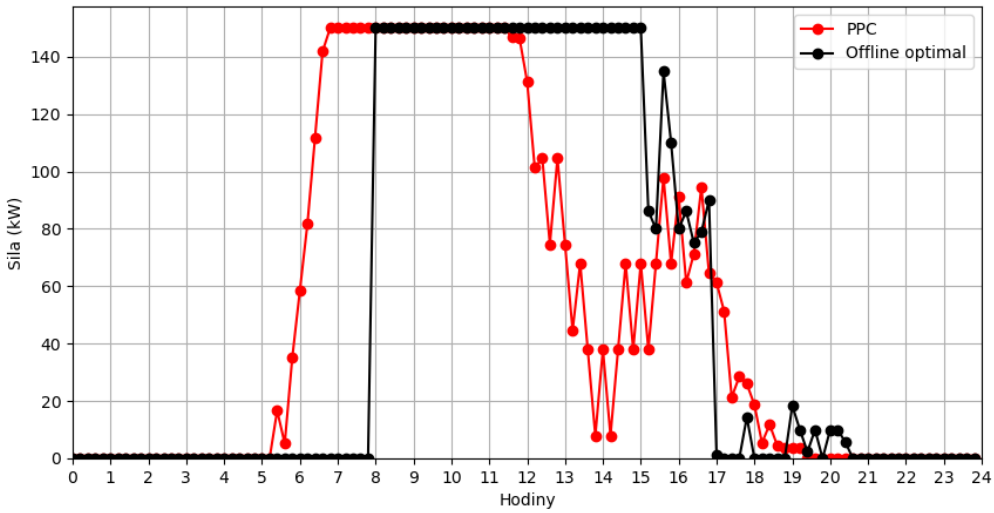
4.3 Opis experimentov

V tejto sekcii opisujeme simulované experimenty, ktoré sme vykonali. Uvádzame aj hyperparametre s ktorými sme v jednotlivých experimentoch pracovali.

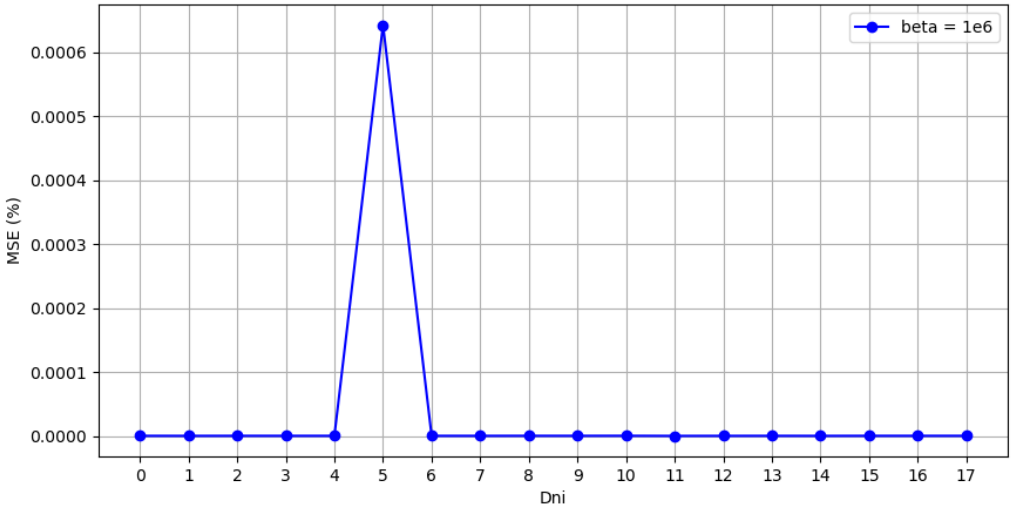
V experimentoch sme nastavili maximálnu mieru sklonu na $30kW$ a maximálnu spotrebu energie na $150kW$. Vyhladzovací parameter β sme nastavili na hodnotu $1e-6$. S vyhladzovacím parametrom $\beta = 1e-6$ dokážeme dodržať obmedzenia optimalizačného problému, ale na úkor minimalizácie nákladov [12]. Množina U_t sa skladá z 10 realizovateľných akcií (energií), medzi ktorými je rovnaký rozostup. Čas medzi susednými časovými krokmi sme nastavili na $\Delta = 0.2$ hodín.



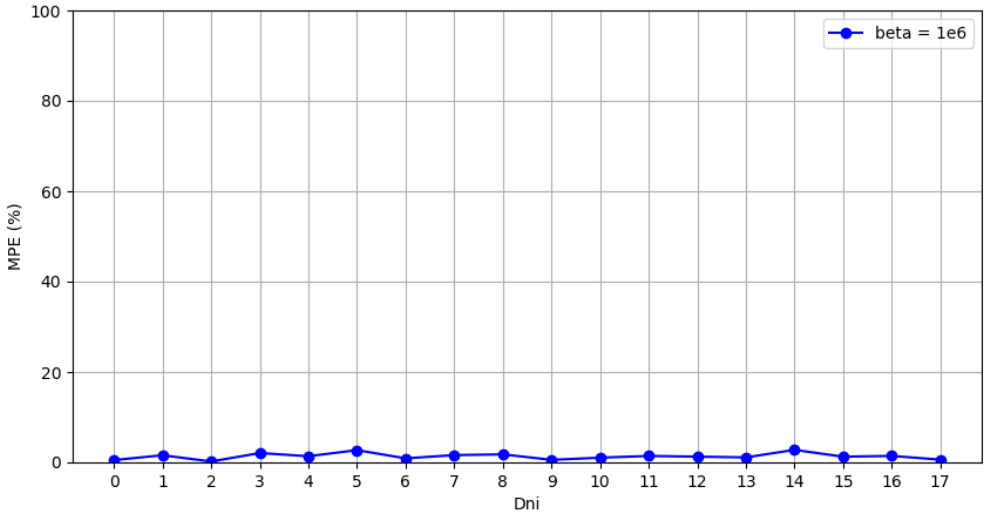
Obr. 4.1: Ukážka nabíjania v nabíjacej stanici Caltech z 13. Decembra 2019.



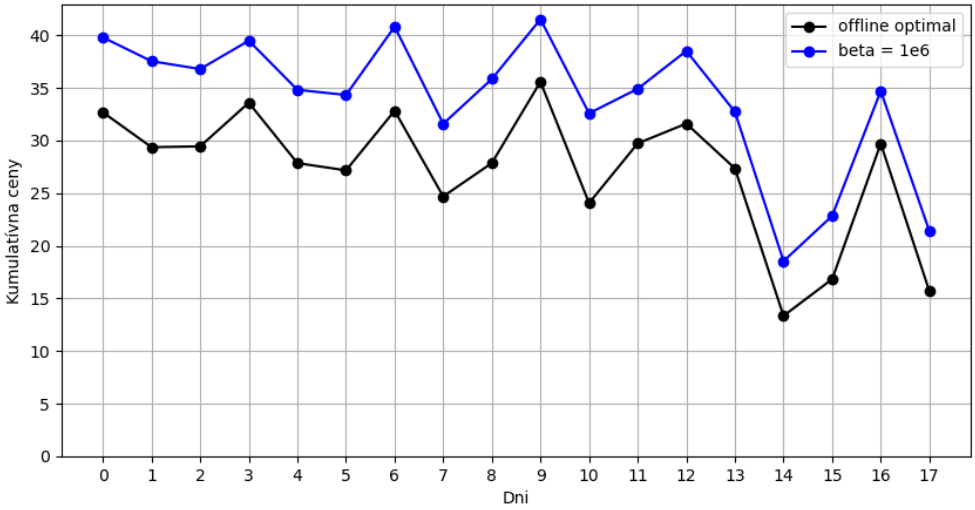
Obr. 4.2: Ukážka nabíjania v nabíjacej stanici jpl z 11. Decembra 2019.



Obr. 4.3: Ukážka nabíjania z dát caltechovskej nabíjacej stanice z 13. Decembra 2019.



Obr. 4.4: Ukážka nabíjania z dát caltechovskej nabíjacej stanice z 13. Decembra 2019.



Obr. 4.5: Ukážka nabíjania z dát caltechovskej nabíjacej stanice z 13. Decembra 2019.

Záver

Prínosov tejto práce je viacero

1. implementácia systému uzavretej slučky. Síce už existuje podobný systém ..., nami implementovaní systém sa odlišuje v odmenách, v generovaní signálu u_t. Celkovo dizajn nami implementovaného systému uzavretej slučky sa trochu odlišuje od uvedeného.

2. implementácia algoritmu smoothed least-laxity-first. Pri našich najlepších vedomostiach sme nevedeli nájsť implementovaný algoritmus smoothed least-laxity-first.

3. Porovnanie riešení optimalizačného problému xxx pomocou systému uzavretej slučky. Prvé riešenie využíva jednoduchý plánovací algoritmus LLF a druhé riešenie využíva jednoduchý plánovací algoritmus sLLF.

implementácia systému uzavretej slučky, ktorá používa algoritmus

Zlepšenie spôsobu učenia prípadne nájdenie optimálnej aproximácie maximálnej entropickej odozvy sú možnosti, pomocou ktorých je možné dosiahnuť lepšie výsledky. Maximálna entropická odozva sa môže použiť aj v iných optimalizačných problémoch dosahujúc viacero cielov nad optimalizáciou nabíjania elektromobilov. My nechávame tieto zlepšenia pre budúci výskum [12, 11].

Literatúra

[1] Moses Kabeyi and Oludolapo Olanrewaju. The use of smart grids in the energy transition. 03 2022.

[2] Santi Martínez, Francesc Sebé, and Christoph Sorge. Measuring privacy in smart metering anonymized data, 2020.

[3] Jixuan Zheng, David Wenzhong Gao, and Li Lin. Smart meters in smart grid: An overview. *2013 IEEE Green Technologies Conference (GreenTech)*, pages 57–64, 2013.

[4] Rajini Karduri. Integrating renewable energy into existing power systems: Challenges and opportunities, 03 2018.

[5] seps. O spoločnosti. `https://www.sepsas.sk/o-nas/o-spolocnosti/`, 2023. Posledný prístup: 3. Decembra, 2023.

[6] John S. Vardakas, Nizar Zorba, and Christos V. Verikoukis. A survey on demand response programs in smart grids: Pricing methods and optimization algorithms. *IEEE Communications Surveys & Tutorials*, 17:152–178, 2015.

[7] Zachary J. Lee, John Z. F. Pang, and Steven H. Low. Pricing EV charging service with demand charge. *Electric Power Systems Research*, 189:106694, 2020.

[8] Jan Stede, Karin Arnold, Christa Dufter, Georg Holtz, Serafin von Roon, and Jörn C. Richstein. The role of aggregators in facilitating industrial demand response: Evidence from germany. *Energy Policy*, 147:111893, 2020.

[9] Pedro Crespo del Granado, Jayaprakash Rajasekharan, Surya Venkatesh Pandiyan, Asgeir Tomasgard, Güray Kara, Hossein Farahmand, and Stefan Jaehnert. Flexibility characterization, aggregation, and market design trends with a high share of renewables: a review. *Current Sustainable/Renewable Energy Reports*, 10(1):12–21, Mar 2023.

[10] Omid Sadeghian, Arman Oshnoei, Behnam Mohammadi-ivatloo, Vahid Vahidinasab, and Amjad Anvari-Moghaddam. A comprehensive review on electric vehicles smart charging: Solutions, strategies, technologies, and challenges. *Journal of Energy Storage*, 54:105241, 2022.

[11] Tongxin Li, Yue Chen, Bo Sun, Adam Wierman, and Steven H. Low. Information aggregation for constrained online control. *Proc. ACM Meas. Anal. Comput. Syst.*, 5(2), June 2021.

[12] Tongxin Li, Bo Sun, Yue Chen, Zixin Ye, Steven H. Low, and Adam Wierman. Learning-based predictive control via real-time aggregate flexibility. *IEEE Transactions on Smart Grid*, 12(6):4897–4913, November 2021.

[13] Tongxin Li, Steven H. Low, and Adam Wierman. Real-time flexibility feedback for closed-loop aggregator and system operator coordination, 2020.

[14] Niangjun Chen, Christian Kurniawan, Yorie Nakahira, Lijun Chen, and Steven H. Low. Smoothed least-laxity-first algorithm for EV charging, 2021.

[15] Linqi Guo, Karl Fredrik Erliksson, and Steven H. Low. Optimal online adaptive electric vehicle charging. *2017 IEEE Power & Energy Society General Meeting*, pages 1–5, 2017.

[16] Ioannis Antonopoulos, Valentin Robu, Benoit Couraud, Desen Kirli, Sonam Norbu, Aristides Kiprakis, D. Flynn, Sergio Elizondo-Gonzalez, and Steve Wattam. Artificial intelligence and machine learning approaches to energy demand-side response: A systematic review. *Renewable and Sustainable Energy Reviews*, 130:109899, 09 2020.

[17] Zachary Jordan Lee. *The Adaptive Charging Network Research Portal: Systems, Tools, and Algorithms*. Dissertation, California Institute of Technology, 2021.

[18] Zachary J. Lee, George Lee, Ted Lee, Cheng Jin, Rand Lee, Zhi Low, Daniel Chang, Christine Ortega, and Steven H. Low. Adaptive charging networks: A framework for smart electric vehicle charging, 2020.

[19] Zhaoxi Liu, Qiuwei Wu, Kang Ma, Mohammad Shahidehpour, Yusheng Xue, and Shaojun Huang. Two-stage optimal scheduling of electric vehicle charging based on transactive control. *IEEE Transactions on Smart Grid*, 10(3):2948–2958, 2019.

[20] Guanchen Zhang, Shaoqing Tim Tan, and G. Wang. Real-time smart charging of electric vehicles for demand charge reduction at non-residential sites. *IEEE Transactions on Smart Grid*, 9:4027–4037, 2018.

[21] Sichen Li, Weihao Hu, Di Cao, Tomislav Dragicevic, Qi Huang, Z. Chen, and F. Blaabjerg. Electric vehicle charging management based on deep reinforcement learning. *Journal of Modern Power Systems and Clean Energy*, 10:719–730, 05 2022.

[22] Mohammad Hajidavalloo. Energy cost minimization in an electric vehicle solar charging station via dynamic programming. *Journal of Computational and Applied Mechanics*, 01 2020.

[23] University of Toronto. Indicator functions, n.d. Accessed: 2024-12-23.

[24] Jean Gallier. Vector norms and matrix norms, 2024. Slides for: Fundamentals of Linear Algebra and Optimization, University of Pennsylvania, August 28, 2024.

[25] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018.

[26] ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY. 1.2 open and closed loop systems, n.d. Accessed: 2024-12-17.

[27] Yorie Nakahira, Niangjun Chen, Lijun Chen, and Steven Low. Smoothed least-laxity-first algorithm for EV charging. 05 2017.

[28] Niangjun Chen, Chee Wei Tan, and Tony Q. S. Quek. Electric vehicle charging in smart grid: Optimality and valley-filling algorithms. *IEEE Journal of Selected Topics in Signal Processing*, 8:1073–1083, 2014.

[29] Zachary J. Lee, Tongxin Li, and Steven H. Low. ACN-Data: Analysis and applications of an open EV charging dataset. In *Proceedings of the Tenth ACM International Conference on Future Energy Systems*, e-Energy '19, page 139–149, New York, NY, USA, 2019. Association for Computing Machinery.

[30] Zachary J. Lee, Sunash Sharma, Daniel Johansson, and Steven H. Low. ACN-Sim: An open-source simulator for data-driven electric vehicle charging research, 2021.

[31] Caltech. ACN-Data. `https://ev.caltech.edu/dataset`, 2023. Accessed: 3rd December, 2023.

[32] Sunash Sharma, Zachary Lee, Daniel Johansson, and liv20. zach401/acnportal: v0.3.3, November 2023. Accessed: December 2023.

[33] Tongxin Li. Enhancing responsible AI for scalable and adaptive EV charging. `http://www.chatziva.com/presentations/PESGM2023TrustworthyAI/TongxinLi_TrustworthyAI_PESGM_2023.mp4`, 2023. Accessed: 2024-12-9.

[34] Akshay Agrawal, Robin Verschueren, Steven Diamond, and Stephen Boyd. A rewriting system for convex optimization problems. *Journal of Control and Decision*, 5(1):42–60, 2018.

[35] Steven Diamond and Stephen Boyd. CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.

[36] Tongxin Li. Information aggregation for constrained online control. Presentation at Conference Name, 2021. Available at: `https://github.com/tongxin-li/Sigmetric2021_slides/blob/main/Sigmetrics2021.pdf`, Accessed: 2024-12-9.

[37] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[38] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

Príloha A: obsah elektronickej prílohy

Príloha B: Používateľská príručka

Implementácia je naprogramovaná v programovacom jazyku python. Odporúčame inštalovanie pycharmu a anacondy na spustenie programu. Cez anacondu treba v tom prípade nainštalovať cvxpy, stable baselines 3 a prípadne ďalšie knižnice.

Program spúštame/testujeme pomocou pythonovských testov.